1 | The mini-project

This mini-project relates to the Mandelbrot set (after the mathematician Benoit Mandelbrot), which leads to compelling two-dimensional fractal patterns. A fractal contains elements of self-similarity when plotted. The Mandelbrot set is a quadratic complex mapping of the type:

$$z_{i+1} = z_i^2 + c, \quad i = 0, 1, \ldots, I-1 \tag{1.1}$$

where $c \in \mathbb{C}$ is a point in the complex plane, and $z_i \in \mathbb{C}$ for $i = 0, 1, \ldots, I$. This provides us with $z_0, z_1, \ldots, z_I$ where the initial condition is $z_0 = 0 + j \cdot 0$ and $z_1, \ldots, z_I$ are referred to as iteratively achieved outputs. In Equation (1.1) the iteration number is $i + 1 \in \{1, 2, \ldots, I\}$ and the total number of iterations is I. For each observed complex point c we then compute I iterations of Equation (1.1). For each complex point c we then determine:

$$\iota(c) = \min T, \quad T = \{\, i \mid |z_i| > T,\ i = 1, 2, \ldots, I\} \cup \{I\} \tag{1.2}$$

where T is a threshold value and the initial condition is always chosen such that $|z_0| \leq T$. From Equation (1.2) we see that $1 \leq \iota(c) \leq I$. So, if $|z_{i+1}| \leq T,\ \forall i = 0, \ldots, I-1$ we obtain $T = \varnothing \cup \{I\} = \{I\}$ meaning that $\iota(c) = I$. Obviously, in a computational implementation we need not compute all I iterations if an $i + 1 < I$ leads to $|z_{i+1}| > T$. We then just set $\iota(c)$ to the smallest $i + 1$ that leads to $|z_{i+1}| > T$. For plotting purposes we then form a simple linear mapping as:

$$M(c) = \frac{\iota(c)}{I}, \quad 0 < M(c) \leq 1 \tag{1.3}$$

The smaller $M(c)$ value we have, the faster that specific complex point c makes $|z_{i+1}|$ in Equation (1.1) increase. In a case with the number of iterations $I = 100$ a value of $0 < M(c) \lesssim 0.1$ indicates an extremely 'active' (unstable) point whereas a value of $M(c) = 1$ indicates a stable point. A point c is said to belong to the Mandelbrot set if $|z_{n+1}|$ remains bounded (finite) for $n \to \infty$.

In this mini-project, the computational task is then to determine $M(c)$ for a c-mesh, which we limit c-wise to $-2 \leq \Re\{c\} \leq 1$ and $-1.5 \leq \Im\{c\} \leq 1.5$. We then select a certain number of points for each of $\Re\{c\}$ and $\Im\{c\}$ as `pre` and `pim`, respectively. The complex plane limited by the mesh is described by a complex matrix C as:

$$C = \begin{bmatrix} -2.0 & \cdots & 1.0 \\ \vdots & \ddots & \vdots \\ -2.0 & \cdots & 1.0 \end{bmatrix} + j \cdot \begin{bmatrix} 1.5 & \cdots & 1.5 \\ \vdots & \ddots & \vdots \\ -1.5 & \cdots & -1.5 \end{bmatrix} \in \mathbb{C}^{pim \times pre} \tag{1.4}$$

You should select `pre` and `pim` according to the computational resources you have available and the desired resolution. A likely number could be something like `pre = 5000` and `pim = 5000`. We use a threshold of $T = 2$. An example is shown in Figure 1.1.

Solution by KP.

The program in the miniproject-1.py computes and benchmarks the **Mandelbrot set** using three different implementations:

1. **Native Python**
2. **NumPy**
3. **Numba**

The goal is both to:

- generate Mandelbrot set data ;)
- compare performance between different implementation styles (this is achieved y measuring the time it take to calculate the set)

---

1. Configuration

At the top of the file, the program defines:

- the complex-plane boundaries
  - `XMIN = -2.0`, `XMAX = 1.0`
  - `YMIN = -1.5`, `YMAX = 1.5`
- the maximum number of iterations
  - `MAX_ITER = 100`
- the tested resolutions
  - `RESOLUTION_RAMP_UP = [256, 512, 1024, 2048, 4096, 8192]`

This means the Mandelbrot set is computed several times at increasing image sizes. This is done to test the limits od each approach. The limit is reach when the calculation takes more than a minute.

---

2. Native Python implementation

`native_python_implementation(...)`

This version uses plain Python loops.

First it calculates the even spread grid for the given range and resolution. for yeac y and x pos this is achieved by that formula:

p_y = ymin + (y / (height - 1)) * (ymax - ymin) (example for y - same for x with min and max y and height params)

Performance note: this is a claculation that is done INDIVIDUALLY for all point (once per row it get the y value x individually for point)

TO see the mandelbrod set, for every pixel:

1. it maps the pixel position to a complex number `c` (c = x + 1j * y)
2. it starts with `z = 0` (for each point: z_n= z_n+1 **2 + c)
3. it repeatedly computes

[z_{n+1} = z_n^2 + c]

4. it stops when either:
    - `|z| > 2`, or
    - `max_iter` is reached

The number of iterations before escape is stored in the output grid.

The slow performance comes from the repeatitive calculations beeing done point by point:

- take one complex number
- update it
- check if it escaped
- repeat

---

3. NumPy implementation

`numpy_implementation(...)`

This version uses vectorized NumPy operations. (several calculations at the same time)

It:

- builds the full complex grid at once using `linspace` and `meshgrid`
- stores all current `z` values in an array
- updates many points simultaneously using masking

this creates an array of masks ([True, True, False]) that allows for easy calculations of the remaining velues in a vestorized matter.

The performance improvments comes from many calculations nbeeing bulk tgehter.

---

4. Numba implementation

`numba_implementation(...)`

This version is the same as native Python version, but it is decorated with:

`@jit(nopython=True)`

Numba compiles the loop-heavy Python code into machine code, which usually gives a large speedup.

This version is useful because it keeps the logic simple and explain stpe by step while improving performance.

---

5. Utility functions

`visualize(...)`

This function plots the Mandelbrot output using `matplotlib`.

It:

- creates a figure
- shows the computed 2D array as an image
- labels the real and imaginary axes
- adds a colorbar

`export_to_csv(...)`

This function saves results to CSV.

It supports two kinds of data:

- **timing results** as tuples
- **Mandelbrot arrays** as integer grids

---

6. Timing and Benchmarking

The script then loops over each resolution in `RESOLUTION_RAMP_UP`.

For each resolution it:

1. computes the Mandelbrot set with native Python
2. computes it with NumPy
3. warms up Numba
4. computes it with Numba
5. stores the execution time for each method

The results are collected in the `results` list.

Results Table

The following table shows the execution time (in seconds) for each implementation across all tested resolutions:

Resolution	Native Python (s)	NumPy (s)	Numba (s)
256×256	0.231	0.040	0.112
512×512	0.852	0.156	0.032
1024×1024	3.406	0.668	0.128
2048×2048	13.876	2.765	0.537
4096×4096	55.840	12.231	2.101
8192×8192	223.529	56.066	8.446

Performance Speedup Factors

To better understand the relative performance gains, the following table shows how much faster NumPy and Numba are compared to native Python:

Resolution	NumPy vs Native	Numba vs Native	Numba vs NumPy
256×256	5.8×	2.1×	0.36×
512×512	5.5×	26.5×	4.8×
1024×1024	5.1×	26.6×	5.2×
2048×2048	5.0×	25.8×	5.1×
4096×4096	4.6×	26.6×	5.8×
8192×8192	4.0×	26.5×	6.6×

Key Observations

Native Python Performance: Native Python shows the slowest performance across all resolutions. At 256×256 resolution, it takes 0.23 seconds. As resolution increases, execution time grows significantly — at 8192×8192, the computation takes over 3minutes. This is expected because every point is processed individually in Python loops, which introduces substantial overhead.

NumPy Performance: NumPy is consistently faster than native Python, providing speedups between 4× and almost 6x depending on the resolution. The speedup is most pronounced at lower resolutions (5.8× at 256×256) and decreases slightly at very high resolutions (4.0× at 8192×8192). Even at the highest resolution tested, NumPy completes in under one minute, making it practical for interactive exploration.

Numba Performance: Numba shows the best overall performance, especially at medium and high resolutions. At 512×512, Numba is already 26.5× faster than native

Python. This advantage is maintained across all tested resolutions, with Numba achieving a consistent 25-26× speedup factor. At 8192×8192, Numba completes in just 8.4 seconds compared to 223.5 seconds for native Python.

JIT Warmup Effect: At 256×256 resolution, Numba appears slower than NumPy (0.112 vs 0.040 seconds). This is because the JIT compilation overhead dominates at very low problem sizes. Once the function is compiled and the problem becomes larger, Numba's compiled code substantially outperforms NumPy's interpreted operations.

7. CSV export

To prove that the results are real the CSV files with results are attahced (the larges set was too big for github and it was removed)